

WEEK 1 — SECURITY

Map any authorized target in an afternoon

ارسم خريطة أي هدف مصرّح به في عصرية

YOUR AI TOOL

Claude

Tuned for Claude — paste each block as a Project instruction or a /command, and let extended thinking expand the checklist before it acts.

مهيئة ل Claude — الصق كل جزء كتعليمات Project أو أمر /، وخذ التفكير الموسّع يوسّع القائمة قبل التنفيذ.

YOUR SYSTEM · BASH

Linux

Commands use bash. Run inside a scoped, authorized environment only.

الأوامر ب bash. شغلها داخل بيئة مصرّح بها فقط.

The Authorized Recon Playbook: Map a Target in an Afternoon

ابدأ هنا. هذا الدليل العملي يعلمك كيف ترسم خريطة كاملة لأي هدف مصرّح لك باختباره — في عصرية واحدة وبمساعدة مساعد ذكي. ستتعلم كيف تنتقل من اسم نطاق واحد إلى صورة منظّمة عن السطح الهجومي، الخدمات، والأدلويات، ثم تحوّلها إلى تقرير مكتوب يمكن تسليمه.

Only test systems you own or hold explicit written permission to test. No authorization, no engagement — full stop.

Reconnaissance is where good security work is won or lost. The aim of this afternoon is not to break anything; it is to *understand* the target so completely that later phases become obvious. You are building a map: every name, every address, every exposed service, ranked by how much it matters. Work calmly, write everything down, and let the AI assistant carry the tedious correlation while you keep judgment in your own hands.

1. Scope and Rules of Engagement

Before a single packet leaves your machine, pin down the boundaries in writing. Recon outside scope is not recon — it is trespass.

- Confirm exactly which domains, IP ranges, and applications are in scope, and note anything explicitly excluded.
- Record the authorization window: start date, end date, and any blackout hours where testing is forbidden.
- Capture contact details for the asset owner and an emergency stop procedure.
- Decide your noise tolerance: are you permitted active probing, or passive-only for now?
- Save all of this in a single engagement file. Every later finding references it.

Treat this step as a contract with yourself. If a discovered asset is not clearly inside the agreed boundary, you flag it and ask — you do not touch it.

2. Passive Footprinting

Start with what the world already knows. Passive footprinting gathers public information without sending traffic that could alert or affect the target.

- Pull the domain's public registration and historical records to understand ownership and age.
- Enumerate DNS thoroughly: A, AAAA, MX, NS, TXT, and especially CNAME chains that reveal third-party services.
- Inspect certificate transparency logs — issued certificates expose subdomains and internal naming conventions teams forget are public.
- Note the email and SPF/DMARC posture; it hints at infrastructure providers and security maturity.
- Look for the organization's public code, job postings, and documents that leak technology choices.

What you are hunting for: the *shape* of the estate. Which cloud providers, which naming patterns, which forgotten staging hosts. Why it matters: these breadcrumbs almost always point to the soft, half-maintained corners that later phases care about most.

PROMPT TEMPLATE — Passive footprint synthesis
You are my recon analyst for an AUTHORIZED engagement.
Scope (in-scope only): {paste scope}

```
Here is raw passive data collected (DNS records, certificate-log
subdomains, registration info):
{paste raw data}
Tasks:
1. Deduplicate and group every hostname under its parent domain.
2. Flag anything that looks like staging, dev, admin, vpn, or legacy.
3. Infer the likely hosting/cloud providers and why.
4. List my top 5 unanswered questions for the next step.
Output a clean table plus a short "what stands out" note.
Do not suggest any action against out-of-scope assets.
```

3. Surface Mapping

Now turn the scattered names into a structured attack surface. The goal is a single inventory you trust.

- Consolidate every discovered host and resolve each to its current IP address(es).
- Separate live hosts from dead records — old DNS entries are common and waste time later.
- Group assets by environment (production, staging, internal-facing) and by owner where known.
- Note which hosts share infrastructure; a shared IP or load balancer changes how you reason about them.
- Mark any asset whose ownership is ambiguous for an explicit authorization check.

The output is an asset register: a living table that everything downstream points back to. A messy surface map guarantees a messy engagement.

4. Service and Technology Identification

With live hosts in hand, identify *what each one runs* — at a careful, permitted level of probing.

- For each live host, determine open, reachable services and the protocols behind them.
- Capture service banners and version hints, but treat versions as leads, not gospel.
- Fingerprint web technologies: server software, frameworks, content platforms, and front-end libraries.
- Identify authentication surfaces — login portals, APIs, admin panels — and how they are exposed.
- Record default or telltale paths that reveal a known platform without intrusive testing.

What to look for: the gap between *intended* exposure and *actual* exposure. Why: an admin panel reachable from the internet, or an unexpected service on a forgotten port, is the kind of finding that defines an engagement.

```
PROMPT TEMPLATE – Service fingerprint review
Context: AUTHORIZED recon. I will paste service/banner/tech-stack
data per host. For each host:
- Summarize what it appears to run and its public role.
- Note version hints and whether they are reliable or guesses.
- Identify the riskiest exposed surface (auth panel, API, file share).
- Rate exposure concern as Low / Medium / High with one-line reasoning.
Keep findings evidence-based; mark anything speculative as "unconfirmed".
Data:
{paste per-host data}
```

5. Prioritizing What Matters

You will end up with far more findings than you can pursue. Prioritization is the senior skill.

- Rank assets by a blend of exposure, sensitivity, and how unusual the finding is.
- Push anything internet-facing with authentication or sensitive data to the top.
- Down-rank well-maintained, hardened, fully-patched production services — they are honest noise.
- Elevate the forgotten things: staging copies, default credentials surfaces, orphaned subdomains.
- For each high-priority item, write one sentence on *why* it is worth deeper, authorized testing.

The deliverable here is a short, ordered shortlist. Reconnaissance that ends in a flat dump of hosts is incomplete; a ranked list with reasoning is what a client or team can act on.

6. From Findings to a Written Report

Recon only counts when someone else can read it and act. Close the afternoon by writing it up.

- Open with scope, authorization reference, dates, and method — so the report is defensible.
- Present the asset inventory and the prioritized shortlist as the centerpiece.

- For each notable finding, state the evidence, why it matters, and a suggested next step.
- Separate confirmed facts from inferences; never overstate certainty.
- End with open questions and recommended follow-up, framed for authorized continuation only.

PROMPT TEMPLATE – Recon report draft

Turn my findings into a professional recon report for an AUTHORIZED engagement. Audience: technical asset owner.

Sections: Scope & Authorization, Method, Asset Inventory,

Prioritized Findings (evidence / impact / next step), Open Questions.

Tone: precise, factual, no hype. Mark inferences clearly.

Findings:

{paste your consolidated notes}

Checklist

- ☒ Written authorization and scope confirmed before any activity
- ☒ Authorization window and blackout hours recorded
- ☒ Passive DNS, certificate logs, and public info gathered first
- ☒ Stray, staging, and legacy hosts flagged for review
- ☒ Live hosts resolved and consolidated into one asset register
- ☒ Services and technologies fingerprinted with confidence noted
- ☒ Findings ranked by exposure, sensitivity, and novelty
- ☒ Out-of-scope assets flagged, never probed
- ☒ Report drafted with evidence, impact, and next steps
- ☒ Confirmed facts separated from inferences throughout